

Anforderungen an die Projektarbeit in Software Engineering 2

Stand: 26.1.2026 FS26

Der benotete Leistungsnachweis im Modul «Software Engineering 2» besteht aus der hier beschriebenen Projektarbeit, welche als Einzelarbeit durchgeführt wird. Die formulierten Anforderungen stellen gleichzeitig die Bewertungskriterien dar, nach denen die Leistungen beurteilt werden.

Das Wichtigste in Kürze

- Es wird ein Prototyp einer Web-Applikation mit Frontend, Backend und Datenbank entworfen und realisiert. Dabei setzen die Studierenden eine eigene Idee um. Die Dozierenden dürfen jedoch bei der Besprechung der Projektidee Änderungen verlangen.
- Die Bewertung berücksichtigt den Projektinhalt (60%), die Dokumentation (15%) und die Präsentation (25%). Nicht erfüllte pass/fail Leistungsnachweise geben ¼ Note Abzug.

Termine

- **1-13. Semesterwoche:** Wöchentliche Abgabe des Übungsprojekts¹. (pass/fail)
- **2 oder 3. Semesterwoche:** Besprechung der Projektidee mit dem Dozenten. Dauer: 10 Minuten. Mitbringen: Use-Case Diagramm und ER-Diagramm. (pass/fail)
- **3. Semesterwoche:** Abgabe von Explore- Create- und Evaluate-Boards. (pass/fail)
- **4. Semesterwoche:** Gegenseitige Beurteilung der Boards. (pass/fail)
- **4. Semesterwoche:** Pitch Probelauf in der KK. Präsenzplicht. (pass/fail)
- **5. oder 6. Semesterwoche:** Pitch der Projektidee in der KK. (benotet)
- **5. und 6. Semesterwoche:** Feedback zu den Pitches der Mitstudierenden in der KK. Präsenzplicht. (pass/fail)
- **6. Semesterwoche:** Abgabe Pitch-Deck. (pass/fail).
- **7. bis 14. Semesterwoche:** Unterstützung bei der Projektarbeit während der KK nach Bedarf.
- **11. Semesterwoche:** Gegenseitiges Feedback zum Projektstand. (pass/fail)
- **Ende 14. Semesterwoche (24.5.26):** Abgabe der schriftlichen Arbeit und der Implementation. Die Details zur Abgabe sind am Ende dieses Dokuments beschrieben.
- **Kalenderwoche 23 (1.6.-5.6.26):** Präsentation des Projekts. Alternativ darf in Ausnahmefällen auch in Semesterwoche 14 präsentiert werden.

Bemerkung: Die genauen Abgabezeitpunkte (Datum und Uhrzeit) sind in Moodle ersichtlich.

Wichtig: Du hast für dieses Projekt 13 Wochen Zeit. Es gibt keine Erstreckung der Abgabefrist für Krankheiten von weniger als zwei Wochen Dauer. Längere Krankheiten müssen bis spätestens zwei Wochen vor Abgabefrist gemeldet und belegt werden. Plane dein Projekt so, dass du es rechtzeitig abgeben kannst. Nicht rechtzeitig abgegebene Projekte werden mit 1.0 bewertet.

¹ Ab Semesterwoche 4 darf alternativ der Inhalt des Übungsprojekts im eigenen Projekt umgesetzt werden.

Technische Anforderungen an das Projekt

- Die Anwendung verfügt über ein Frontend basierend auf SvelteKit.
- Die Anwendung verfügt über ein Backend basierend auf Spring Boot.
- Das Spring-Backend verwendet MongoDB auf Atlas für die Persistenz.
- Das Frontend greift ausschliesslich via Spring-Backend auf die Datenbank zu.
- Die Anwendung ist auf Azure App Service deployed.
- Die Anwendung enthält mit JUnit erstellte Tests. Assertions, parametrisierte Tests, Exception Testing und Mockito werden sinnvoll verwendet. Möglichst alle Endpoints werden mittels @SpringBootTest und @MockMvc rollenabhängig getestet und das JSON-Resultat (sofern vorhanden) sinnvoll geprüft.
- Die Anwendung verfügt über mindestens zwei authentifizierte Benutzerrollen mit unterschiedlichen Berechtigungen und einem Login basierend auf Auth0. Frontend und Spring-Backend sind beide durch den Login geschützt.
- Die Anwendung greift via Spring AI-API auf ein KI-Modell zu und verwendet dieses sinnvoll.
- Das Datenmodell enthält mindestens drei Entitätstypen mit Beziehungen. Die drei Entitäten werden in der DB in separaten Collections gespeichert. Mindestens eine Entität durchläuft mehrere Zustände. Mindestens eine Entität ist einem User zugeordnet und kann nur von diesem bearbeitet werden. Alle Entitäten können über das Frontend erstellt und wenn fachlich sinnvoll bearbeitet und gelöscht werden.

Anforderungen an das Vorgehen und an die verwendeten Tools

- Das Deployment auf Azure App Service erfolgt mit Dockerfile und GitHub Action. Erfolgreiche Logs der Deployments liegen im GitHub-Repository vor.
- Die JUnit-Tests werden automatisch mittels GitHub Actions ausgeführt. Die Ausführungsprotokolle sind im GitHub-Repository verfügbar. Die JaCoCo-Testabdeckung wird gemessen und sowohl als Status-Badge auf der Startseite des Repositories angezeigt als auch als HTML-Bericht abgegeben. Die Abdeckung ist möglichst hoch (ca. 90%, entspricht «sehr gut»).
- Die API (Endpoints) ist mit Postman dokumentiert und veröffentlicht.
- Die Anwendung wird auf einem privaten GitHub-Repository entwickelt.
- Anforderungen für den Umgang mit GitHub:
 - In GitHub wird ein SCRUM-Board erstellt und verwendet.
 - Die Umsetzeng erfolgt in mehreren Sprints. Die Sprints sind in GitHub als Iterations abgebildet. Die Sprints haben eine sinnvolle Dauer.
 - Arbeitspakete sind in GitHub als Issues dokumentiert. Issues haben eine Beschreibung und überprüfbare Anforderungen. Es werden mindestens drei unterschiedliche Labels verwendet. Issues sind einem Sprint oder dem Backlog zugeordnet. Issues durchlaufen im SCRUM-Board die Spalten Ready, In Progress und Done. Abgeschlossene Issues haben den Zustand Closed.

Optionale Anforderungen

Durch das Umsetzen der folgenden Anforderungen können die Erwartungen übertroffen werden:

- Codeanalyse mit SonarQube.
- Komplexes Datenmodell.
- Komplexes Frontend.
- Zugriff auf Drittsysteme.
- Backend mit MCP-Server.
- Komplexe Benutzerverwaltung.
- Komplexe Abfragen auf der Datenbank.
- Detaillierte Dokumentation auf GitHub (Issues etc.).
- Es werden mehrere Branches sinnvoll verwendet.
- End-to-End Tests der Anwendung.
- Weitere Erweiterungen können direkt mit den Dozierenden vereinbart werden.

Anforderungen an die schriftliche Dokumentation

Die Dokumentation befindet sich im GitHub-Repo im Dokument README.md auf der obersten Ebene des Projekts. Das Dokument basiert auf der Moodle-Vorlage. Bilder sind im Ordner /doc abgelegt und in die MD-Dateien eingebunden. Die Kapitel sowie der zugehörige Inhalte sind unten aufgeführt.

- **Einleitung:**
 - Explore-, Create- und Evaluate-Boards
 - Diskussion des Feedbacks aus dem Pitch (bezogen auf Projekthalt)
- **Anforderungen:**
 - Use-Case Diagramm
 - Use-Case Beschreibung (wie in RE gelernt)
 - Fachliches Datenmodell (ER-Modell), Beschreibung der Zustände
 - Mockup oder Skizze des UIs
- **Implementation:**
 - Beschreibung des Frontends mit Screenshots der fertigen Applikation. Alle Teile des GUIs, die bewertet werden sollen, müssen abgebildet sein.
 - Aufgaben und Funktionen des eingebundenen KI-Modells.
 - Umgesetzte optionale Anforderungen mit Beschreibung.
- **Fazit**
 - Stand der Implementation, nächste Schritte (mit Referenz auf den Backlog).

Die Dokumentation sollte dem tatsächlich umgesetzten Projekt entsprechen. Boards, Use-Case Diagramme etc. dürfen im Verlauf des Projekts überarbeitet werden.

Anforderungen an den Pitch

Der Pitch dauert drei Minuten und basiert auf dem Inhalt der Boards. Der Pitch soll sich an mögliche Investoren oder Institutionen richten, welche das Projekt unterstützen oder finanzieren könnten. Die Mitstudierenden geben Feedback. Der Pitch wird bewertet (20% von Teil Präsentation, 5% der Gesamtwertung). Slides dürfen verwendet werden. Slides mit mehr als 5 Wörtern sind verboten. Der Pitch behandelt die folgenden Fragen:

- Was ist die Idee? (möglichst in einem Satz)
- Wer ist die Zielgruppe? (kurz und präzise)
- Welches Problem der Kunden soll gelöst werden? (Beispiel machen, illustrieren)
- Wie soll das Problem gelöst werden?
- Worin liegt der Mehrwert für die Kunden? (Wertversprechen)
- Wie lässt sich mit der Idee Geld verdienen?

Anforderungen an die Präsentation

Die Abschlusspräsentation dauert 10 Minuten. Anschliessend werden die Studierenden durch die Dozierenden während 10 Minuten zur Arbeit befragt. Folgende Elemente werden während der Präsentation erwartet:

- Ziel der Anwendung: Problem, Nutzergruppe, Lösungsansatz (maximal 1 Minute)
- Demonstration der Anwendung (Live, kein Video, mindestens zwei Rollen)
- Projektstand (maximal 1 Minute)

Abgabe der schriftlichen Arbeit und der Implementation

Am Semesterende müssen alle Dozierenden

- als Collaborators des GitHub-Repos (Repo->Settings->Collaborators) **und**
- als Collaborators beim GitHub-Project (Project->Settings->Manage Access)

hinzugefügt werden (David: DavidZhaw, Adrian: mosazhaw, Max: mmeisterhans, Benjamin: bkuehnis). Über Moodle müssen die folgenden Dokumente abgegeben werden:

- Zip-Ordner des GitHub-Repos (Download von GitHub)
- Zip-Ordner mit dem Namen jacoco.zip, enthält das Verzeichnis «target\site\jacoco» mit den generierten Reports der Testabdeckung
- Zip-Ordner mit dem Namen surefire-reports.zip, enthält das Verzeichnis «target\surefire-reports» mit einer *.txt-Datei pro Testklasse
- URL des GitHub-Repos
- URL der deployten Applikation
- Pro Benutzerrolle je eine Login und Passwort für die deployte Applikation
- URL der Postman API-Dokumentation ([https://documenter.getpostman.com/...](https://documenter.getpostman.com/))
- URL der Roadmap-History (GitHub)
- URL des Charts mit Iterations und Labels (GitHub)
- URL von SonarQube (falls vorhanden)